M-ADSAI-ZM002-26

Reachy Mini: GPU Acceleration Research

Caio De Alcantara Santos (2604500), Cas Limpens (2400410), Donné Deguelle (2301667) and Gustavo Gonçalves Da Costa (2604824).

Contents

Introduction 3
Purpose 3
Scope 3
Research Questions 4
Methodology 5
Hardware and Technical Context 5
GPU Implementations 6
GPU Implementation 1: MoveNet with LiteRT and WebGPU 6
GPU Implementation 2: Native BlazePose with LiteRT and WebGPU 7
Shared LiteRT WebGPU Runtime Failure 8
Future Development and Recommendations 9
Conclusion 10
References 10

Introduction

Purpose

The purpose of this research is to investigate the feasibility of using the integrated GPU of the Reachy Mini to accelerate AI inference, with a particular focus on human pose estimation. This research builds upon the earlier Reachy Mini: Hardware & Technical Capability Assessment (De Alcantara Santos, Limpens, Deguelle, & Gonçalves Da Costa, 2026), which examined the robot's hardware specifications, computing capabilities, available software interfaces and suitability for running AI models locally. This research builds upon those findings by focusing specifically on GPU acceleration. Different frameworks and implementations are investigated to determine whether the Broadcom VideoCore V3D GPU can be used for machine learning inference and which technical limitations prevent successful execution.

Scope

The scope of this research is GPU acceleration of human pose estimation on the Reachy Mini. The goal of the research is not to build a complete GPU-accelerated application. It is rather an attempt to document the process of development, establish how far each implementation goes and understand what needs to be done for successful GPU inference in the future.

Research Questions

The following research questions define the focus of this research.

Main Question

“To what extent is it possible to accelerate human pose estimation on the Reachy Mini using its integrated Broadcom VideoCore V3D GPU, and what technical limitations prevent successful GPU inference?”

Subquestions

1. What are the hardware capabilities and limitations of the Reachy Mini's Raspberry Pi Compute Module 4, and which GPU acceleration frameworks are suitable for this hardware?
2. How can LiteRT and WebGPU be configured and cross-compiled for ARM64 to run MoveNet and BlazePose on the Reachy Mini?
3. What are the differences between the MoveNet LiteRT/WebGPU implementation and the native C++ BlazePose LiteRT/WebGPU implementation?
4. What caused the AES-related runtime failure in both LiteRT/WebGPU implementations, and how was the problem identified?
5. What technical improvements and development steps are required to achieve successful GPU inference on the Reachy Mini in future work?

Methodology

The research has been conducted by iterative implementation, cross-compilation and testing on the Reachy Mini. The GPU applications have been prepared initially on the Ubuntu x86_64 development machine using ARM64 cross-compilation toolchain. Then the resulting binaries and libraries have been tested on the Reachy Mini. In case of failure of GPU initialization additional diagnostics have been performed to find out the processor capabilities, binary instructions, shared library dependencies and runtime loading.

Hardware and Technical Context

The capabilities of the Reachy Mini's hardware were analyzed thoroughly in a previous work entitled Reachy Mini: Hardware & Technical Capability Assessment (De Alcantara Santos, Limpens, Deguelle, & Gonçalves Da Costa, 2026). The robot uses a Raspberry Pi Compute Module 4 with a Broadcom BCM2711 quad-core Cortex-A72 processor, 4 GB of RAM and an integrated Broadcom VideoCore VI GPU (Raspberry Pi, sd). The robot also has IMX708 camera, which serves as a source of video input needed for pose estimation. The prior work also describes other components that can be found on the robot such as sensors, cameras, software development kits, APIs, and networking capabilities.

In this work, the GPU architecture and its software compatibility are the crucial part. The VideoCore GPU can be accessed through Mesa graphics stack and V3D driver(Mesa Project, sd). As the hardware doesn't support NVIDIA CUDA, the alternatives like LiteRT and WebGPU have been analyzed. The following sections describe these technologies and the problems arising from the attempt to use them on the Reachy Mini.

GPU Implementations

GPU Implementation 1: MoveNet with LiteRT and WebGPU

The first dedicated GPU approach focused on MoveNet Lightning INT8 (STMicroelectronics, sd). It was supposed to employ the same model as the future working proof of concept for CPU but with the replacement of the XNNPACK CPU backend with GPU acceleration of the model provided by LiteRT. The latter is Google's runtime for on-device machine learning inference with the APIs for loading models and choosing hardware accelerators supported (Google AI Edge, sd). The envisioned architecture involved feeding the camera into the Python application, which was supposed to preprocess the frames into a 192 × 192 RGB tensor, pass it to the native ARM64 LiteRT runner and get the keypoints to draw a skeleton on top of the camera image.

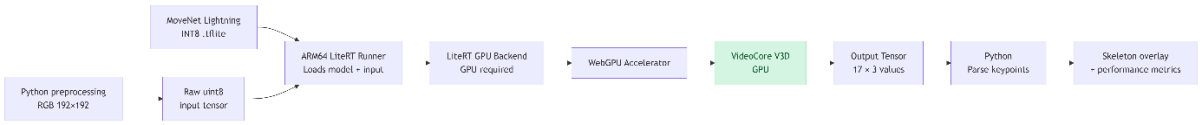


Figure 1 Intended architecture of the MoveNet LiteRT/WebGPU implementation on the Reachy Mini

The LiteRT runtime was built cross-platform on an Ubuntu 24.04 x86_64 development machine using the aarch64-linux-gnu toolchain. In particular, an initial problem arose due to the attempt to run the ARM64 version of the FlatBuffers compiler (flatc) on the x86_64 host platform. It led to an Exec format error. The problem was solved by compiling flatc as a native x86_64 host tool and configuring the build to use that one. With this fix in place, the ARM64 runner was successfully built. It resulted in the files including the LiteRT runner, libLiteRt.so, the prebuilt libLiteRtWebGpuAccelerator.so and the MoveNet INT8 model.

The Python application was then hooked up to the runner. GPU acceleration was explicitly requested in the accelerator configuration including --accelerator=gpu, --gpu_backend=webgpu and --require_registered_accelerator=gpu options. The last one was crucial since it avoided silent falling back to the CPU.

The main problem was GPU initialization. The failure occurred during the first GPU registration check when the MoveNetLiteRT initialization routine was invoked. Specifically, the ARM64 LiteRT runner tried to load the prebuilt WebGPU accelerator library, which was compiled with the AES instructions not supported by the Reachy Mini's Cortex-A72 processor. It caused SIGILL ("illegal instruction" error), leading to failure of the initialization routine. Since the application checks that before connecting to the Reachy Mini camera, the camera processing, image preprocessing and MoveNet inference were not even attempted.



Figure 2 LiteRT WebGPU initialization failure during accelerator registration

GPU Implementation 2: Native BlazePose with LiteRT and WebGPU

The second GPU implementation used the BlazePose Full framework and followed another strategy compared to the MoveNet implementation. Instead of using an existing command-line runner, a custom native C++ inference pipeline was implemented to have greater control over the model execution. The implementation used the original MediaPipe Pose Landmarker Full model bundle, which consisted of two separate TFLite models: a pose detector and a pose landmark model (Google AI Edge, sd). The detector locates a person and defines a region of interest (ROI), and then the landmark model infers the body pose. The original pose_landmarker_full.task bundle was separated into pose_detector.tflite and pose_landmarks_detector.tflite. Both models were loaded separately using LiteRT and set up to run with the GPU backend. The intended processing pipeline was to take a frame from the Reachy Mini camera, transfer it from Python to the native C++ library through ctypes, run the detector on the GPU, calculate the ROI in C++ and then run the landmark model on the GPU. After that, the resulting landmarks should be transferred back to Python to visualize the skeleton and run benchmarking.



Figure 3 Native BlazePose inference architecture using two LiteRT GPU models and a custom C++ processing pipeline

The native implementation was split into the Python interface and the C++ processing part. The Python adapter, which was implemented in gpu_models/__init__.py, loads libblazepose_gpu.so through ctypes and invokes the exported C functions. The C++ library provides the API for creating the GPU pipeline, processing an image, getting the error information and destroying the allocated resources. This allowed to keep using the existing Python benchmark while providing the new implementation, without modifying the original CPU-based one. At the initialization stage, the C++ code creates a LiteRT environment and the GPU execution options. It explicitly enables the GPU acceleration using SetHardwareAccelerators(HwAccelerators::kGpu). Then both the detector and landmark models are loaded using CompiledModel::Create(). Their input and output tensor buffers are created once and stored in the native BlazePoseGpu object. This means that the models and their execution contexts are supposed to remain loaded between camera frames, and not to be recreated at every inference.

At the inference stage, Python converts each camera frame from BGR to RGB and sends the resulted image data to the native library. In C++, the image is resized to 224 × 224 pixels using the letterboxing technique and normalized to the range [-1, 1]. Then the detector runs through LiteRT with the GPU backend. The output includes detection boxes and confidence scores, which are then copied to the C++ memory and further processed using the anchor decoding and NMS. The top-scoring detection is used to calculate the ROI, which consists of the position, scale and rotation of it. Then the region of the image is cropped and rotated to a 256 × 256 input of the landmark model. This second model also runs through LiteRT GPU backend. The output is read back to the C++ memory, and then the coordinates of the first 33 public pose landmarks are converted back to the original image coordinates. Finally, the native library sends the coordinates of the landmarks along with the confidence values to Python, where the skeleton is

drawn on the camera image. The benchmark also collects inference latency, FPS, CPU and RAM usage.

aarch64-linux-gnu-g++ cross-compiler. CMake was configured with CMAKE_SYSTEM_PROCESSOR=aarch64 and the corresponding ARM64 compiler. The compilation depended on the existing LiteRT source and build environment located under ~/LiteRT, including the already built ARM64 LiteRT libraries. An initial error during compilation was related to the aggregate initializer used for initializing the BlazePoseGpu structure with the wrong number of fields. The structure needed an explicit constructor, which accepts a LiteRT environment and two compiled models. This was fixed by using the constructor directly and moving the model objects with std::move(). After this modification, the compilation finished successfully producing libblazepose_gpu.so and blazepose_gpu_inspector. Both binaries were later verified using the Linux file command and proved to be compiled for ARM aarch64.

After the successful cross-compilation, the next step was to test the GPU implementation directly on the Reachy Mini. For this purpose, a separate executable blazepose_gpu_inspector was built. It initializes the LiteRT environment, requests the GPU acceleration, tries to create a CompiledModel for each of the two BlazePose models and checks whether the tensor buffers use the WebGPU memory. The Python benchmark uses almost the same initialization sequence. First, it loads the native library through ctypes and calls blazepose_gpu_create(). This function creates the LiteRT environment, sets up the GPU execution and tries to initialize the detector and landmark models. Importantly, this initialization happens before the benchmark starts its camera warm-up and frame-processing loop. At the first GPU model initialization, LiteRT tries to load the pre-built libLiteRtWebGpuAccelerator.so library. The same AES-related illegal-instruction error occurred, which was observed in the MoveNet implementation. The Cortex-A72 processor of the Reachy Mini does not support these instructions of this binary, so the process terminates before the first CompiledModel can be created successfully. As a consequence, neither detector nor landmark models reach the inference stage, and no GPU tensor buffers or pose landmarks are generated. The successful ARM64 compilation thus proves that the custom application and its native binaries can be built for the Reachy Mini, but it doesn't prove that the whole GPU pipeline can run on it.



Figure 4 BlazePose LiteRT/WebGPU initialization failure caused by unsupported AES instructions on the Reachy Mini

Shared LiteRT WebGPU Runtime Failure

One of the important discoveries made in this case was that the implementation of MoveNet and BlazePose failed not due to two distinct issues related to the model. In both implementations, the pre-built LiteRT WebGPU accelerator was used - libLiteRtWebGpuAccelerator.so. This was checked by comparing the SHA-256 hashes of the accelerator files in the project, and the hashes matched, meaning that the two implementations have one and the same dependency runtime. The capabilities of the Reachy Mini processor were determined with /proc/cpuinfo command, and the output showed no support of AES and PMULL

instruction sets. Using objdump, it was seen that the library had the instructions such as aesd, aese, aesmc and aesimc, which require certain processor features absent in the target CPU architecture. Although the library is compiled for ARM64, its instruction requirements are not compatible with the specific Cortex-A72 configuration.

This was confirmed by a more direct Python ctypes.CDLL() test - the test tried loading the WebGPU library without initializing the camera, loading the model and conducting inference, but resulted in the same fatal error. Checking the WebGPU library with ldd showed that the standard dependencies of a shared library were correctly loaded, but the main libLiteRt.so library did not exhibit the problem with the AES instruction. This way, the incompatibility was isolated to the supplied WebGPU accelerator.

This is why the problem was not fixed with a change of the Python application or rebuilding the native runner, because the problem with the accelerator was due to the fact that it was supplied as a precompiled library. The LiteRT and WebGPU implementations therefore halted at the same initialization point. The next step would be to rebuild or replace the accelerator and its dependencies compiled with appropriate compiler settings for Cortex-A72. Still, even after successful library loading, WebGPU device selection, model loading and inference will have to be checked.

Future Development and Recommendations

In order to move forward with the LiteRT/WebGPU projects, we propose that the WebGPU accelerator and its dependant components be recompiled for the Cortex-A72. This way, all the necessary ARM64 components from the current implementation would be used, while only the pre-built accelerator would need to be reworked. Firstly, the version of the LiteRT source code needs to be found, then the accelerator should be recompiled under the right ARMv8 settings. The resulting binaries need to be checked for unsupported AES instructions (this can be done using objdump). It would also be helpful to check whether the accelerator works independently before trying to run a full pose estimation app using it. For future developers, it would be useful to make sure that the library loads, that the GPU accelerator is registered by LiteRT and that at least a simple model can be initialized and executed. This way, problems associated with model execution and camera initialization would not be confused with each other.

Conclusion

The greatest success was reached while working on the two LiteRT/WebGPU approaches. In particular, the MoveNet approach succeeded in building an ARM64 LiteRT runner and including it in a Python project. The second approach managed to go further in terms of native implementation and created a custom inference pipeline in C++, ARM64 shared library and GPU Inspector. At the same time, both LiteRT/WebGPU approaches stopped at the GPU initialization stage since they used the same prebuilt WebGPU accelerator which had instructions supported by the AES algorithm which is incompatible with the Cortex-A72 processor of the Reachy robot. It was proved by performing CPU feature inspection, analyzing the binary code and an additional library loading test. Thus, the primary output of the current study is a clear diagnosis of the LiteRT/WebGPU incompatibility issue and a technical base for future research. Modification of the WebGPU accelerator with appropriate compiler options is the most promising way to go forward.

References

Mesa Project. (n.d.). *V3D*. Retrieved from Mesa 3D Graphics Library: https://docs.mesa3d.org/drivers/v3d.html

De Alcantara Santos, C., Limpens, C., Deguelle, D., & Gonçalves Da Costa, G. (2026). *Reachy Mini: Hardware & Technical Capability Assessment*.

Google AI Edge. (n.d.). *Getting started with LiteRT*. Retrieved from Google for Developers: https://developers.google.com/edge/litert/overview

Google AI Edge. (n.d.). *Pose landmark detection guide*. Retrieved from Google for Developers: https://developers.google.com/edge/mediapipe/solutions/vision/pose_landmarker

Raspberry Pi. (n.d.). *Compute Module hardware*. Retrieved from Raspberry Pi Documentation: https://www.raspberrypi.com/documentation/computers/compute-module.html

STMicroelectronics. (n.d.). *MoveNet*. Retrieved from GitHub: https://github.com/STMicroelectronics/stm32ai-modelzoo/tree/main/pose_estimation/movenet